13. Git Temelleri & Akıllı Commit Stratejisi: AI ile Anlamlı Geçmiş

'WIP', 'Fixed Bug', 'asdasd' Commitlerinden Kurtulma Rehberi
Bölüm 3 • Zorluk Derecesi: Temel / Orta Seviye • Tahmini Okuma Süresi: 12 dk

---

> 🎙️ **NotebookLM & Audio Overview Podcast Notları**
>
> **Podcast Tartışma Sorusu:**
> *"Git geçmişi neden bir şirketin en değerli dokümantasyonudur ve yapay zeka ile Conventional Commits standardı nasıl otomatikleştirilir?"*
>
> **Bölümden Çıkarılacak 3 Altın Cümle:**
> - 📌 Git commit mesajları sadece ne yaptığınızı değil, o kararı neden aldığınızı anlatmalıdır. - 📌 Conventional Commits (feat, fix, refactor, chore) standardı, yapay zekanın kod tabanını daha iyi anlamasını sağlar. - 📌 AI destekli Git hook'ları (Husky + Git hooks) ile kalitesiz commit mesajları daha terminalden çıkmadan engellenebilir.

📖 1. Giriş Hikayesi & Günlük Hayat Analojisi

Bir adli tıp uzmanının suç mahallini incelediğini düşünün. Masanın üzerinde bir günlük var; ancak sayfaların hepsinde sadece *"Bir şeyler yaptım"*, *"Düzeltme"*, *"Son hali"* yazıyor. Katili bu notlarla bulabilir misiniz? İmkansız!

İşte canlı sunucunuz gece saat 03:00'te çöktüğünde ve acil geri alma (rollback) yapmanız gerektiğinde, o 50 adet *"fixed bug"* veya *"asdasd"* yazan commit mesajları tam olarak o işe yaramaz günlüktür.

Oysa *"fix(auth): prevent token expiration crash by validating clock skew"* yazan bir commit, sistemi saniyeler içinde kurtarır.

Yapay zeka sayesinde artık harika commit mesajları yazmak 2 saniye sürüyor; tembellik için hiçbir mazeret kalmadı.

🧠 2. AI Çağında Yeni Zihniyet Modeli

Conventional Commits Standardı Formülü:

```
<tür>(<kapsam>): <kısa özet (emir kipi)>

[İsteğe bağlı: Neden bu değişiklik yapıldı? (Motivation & Why)]
[İsteğe bağlı: Varsa kırıcı değişiklik (BREAKING CHANGE)]

Türler:
- feat: Yeni bir kullanıcı özelliği
- fix: Bir hatanın düzeltilmesi
- refactor: Davranışı değiştirmeden kodu temizleme
- test: Test ekleme veya düzeltme
- chore: Bağımlılık güncellemesi, yapılandırma
```

🤖 3. AI Prompt Laboratuvarı (Kopyala-Yapıştır Hazır Prompt)

Aşağıdaki prompt şablonunu Cursor, Claude Code, ChatGPT veya Copilot ortamınızda doğrudan kullanabilirsiniz:

````
# ROL: Git ve Sürüm Kontrol Uzmanı
Aşağıda `git diff` çıktım bulunmaktadır.

## GİRDİ (git diff):
```diff
diff --git a/src/auth.ts b/src/auth.ts
--- a/src/auth.ts
+++ b/src/auth.ts
@@ -12,3 +12,6 @@ export function verifyToken(token: string) {
+  if (!token || token.trim() === '') {
+    throw new InvalidTokenError('Token boş olamaz');
+  }
````

GÖREV:

Bu diff için Conventional Commits standardına tam uyumlu: 1. Başlık satırı (Title - en fazla 50 karakter) 2. Açıklama gövdesi (Body - neden yapıldığı ve çözülen güvenlik riski) çıkar.
````

````
---

## 💻 4. Kod İncelemesi: Acemi / Halüsinasyon vs. Üretim Kalitesi

#### ❌ Acemi / Kontrolsüz AI Çıktısı (Riskli):
```bash
# Kötü Alışkanlık: Anlamsız, tek satırlık acele commit
git add .
git commit -m "auth duzeltildi"
git push origin main
````

✅ Kıdemli Mühendis Tarafından İyileştirilmiş Sürüm (Güvenli & Temiz):

```
# AI Destekli Profesyonel Conventional Commit
git add src/auth.ts
git commit -m "fix(auth): validate empty tokens before verification

Guard against empty and whitespace-only authorization tokens.
Prevents unhandled runtime crashes and rejects malicious empty bearer headers
at the perimeter. Closes #104."

# CLI AI Aracıyla Tek Komutta Üretmek:
# npx czg veya aicommits kullanarak otomatik üretin
```

💡 **Kod Analizi:** Kıdemli commit mesajı; hatanın kapsamını (`auth`), eylemi (`validate`), gerekçeyi (çökme ve güvenlik açığı engelleme) ve ilişkili bilet numarasını (`#104`) içerir.

⚠️ 5. Junior Tuzakları & Kıdemli Bakış Açısı

- **Tuzak:** Tek bir devasa commit içine 20 farklı ilgisiz işi koyup 'tüm işler bitti' yazmak.
  - **Çözüm:** Atomik commit prensibini uygulayın; her commit tek bir mantıksal değişikliği temsil etmelidir.
- **Tuzak:** Doğrudan `main` veya `master` branch'ine commit atmak.
  - **Çözüm:** Daima `feat/kullanici-adi` veya `fix/sorun` şeklinde bir feature branch açarak çalışın.
- **Tuzak:** Commit mesajını geçmiş zaman ('düzeltildi') ile yazmak.
  - **Çözüm:** Emir kipi ('düzelt', 'ekle' / 'fix', 'add') kullanarak yazın.

🎬 6. Udemy Video Dersi Taslağı & Slayt Maddeleri

📽️ Ders Başlığı: *Ders 13: Git Ustalığı - AI Destekli Anlamlı Commitler*
```

- 🖥️ **Slayt 1:** Neden kötü commit mesajları şirketlere milyonlarca dolar kaybettirir?
- 🖥️ **Slayt 2:** Conventional Commits standardının anatomisi.
- 🖥️ **Slayt 3:** Atomik commit felsefesi: Küçük, odaklı, geri alınabilir adımlar.
- 🖥️ **Slayt 4:** AICOMMITS ve IDE eklentileriyle tek tuşla commit mesajı üretmek.

Canlı Kodlama / Ekran Kaydı Senaryosu:
Bir dosyada bilinçli bir değişiklik yapın. Terminalde `git diff` alın. Cursor'ın entegre AI commit butonuna veya `aicommits` CLI aracına basarak diff'i analiz ettirip mükemmel bir Conventional Commit başlığı ürettirin.

🎯 7. Günün Uygulamalı Görevi & Mini Sınav

🛠️ Günün Görevi:

Bugünden itibaren yapacağınız sonraki 5 commit için Conventional Commits formatını uygulayın (Örn: `feat:` , `fix:` , `refactor:`).

🧠 Bölüm Sonu Kontrol Testi:

Soru 1: Conventional Commits standardında yeni bir kullanıcı özelliğini sisteme eklerken hangi ön ek kullanılır? - A) fix: - B) feat: - C) chore: - D) docs: > **Doğru Yanıt:** B > **Açıklama:** `feat` (feature) sisteme yeni bir kullanıcı özelliği veya fonksiyon eklendiğini belirtir.

Soru 2: Atomik commit (Atomic Commit) yapmanın birincil teknik faydası nedir? - A) Sabit diski daha hızlı çalıştırması - B) Bir hata durumunda sadece o küçük değişikliği `git revert` ile sistemi bozmadan geri alabilmek - C) Sadece Python projelerinde zorunlu olması - D) GitHub yıldız sayısını otomatik artırması > **Doğru Yanıt:** B > **Açıklama:** Atomik commitler sayesinde bir hata tespit edildiğinde 1000 satırlık devasa bir blok yerine sadece sorunlu 5 satırlık commit güvenle geri alınabilir.